Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Технологий Искусственного Интеллекта

Дисциплина:

«Инженерия данных»

ПРАКТИЧЕСКАЯ РАБОТА № 3

«Аналитика ИТ-вакансий в объектном хранилище: MapReduce и DuckDB»

Вариант 1: Программист, разработчик на 1С

Выполнила:

Андреева Наталия Леонидовна,

студентка группы J3310

Цель работы

Закрепить навыки работы с объектными хранилищами и распределенными вычислениями, а также на сформировать умения проводить сравнительный анализ производительности различных подходов к обработке данных. Развернуть полноценный аналитический пайплайн: от подготовки инфраструктуры и загрузки данных до извлечения бизнес-инстайтов и строгого профилирования ресурсов. Вариант 1: «Программист, разработчик на 1С».

Артефакты выполнения

1.1. Подготовка окружения и хранилища (этап 1).

Использовала хранилище RustFS из практической работы № 2: контейнер rustfs, S3 API на порту 9100, веб-консоль на порту 9101, ключи доступа rustfsadmin. Перед началом проверила состояние контейнера и эндпойнт /health.

```
docker ps --filter name=rustfs
curl.exe -s -o NUL -w "%{http_code}" http://localhost:9100/health
```

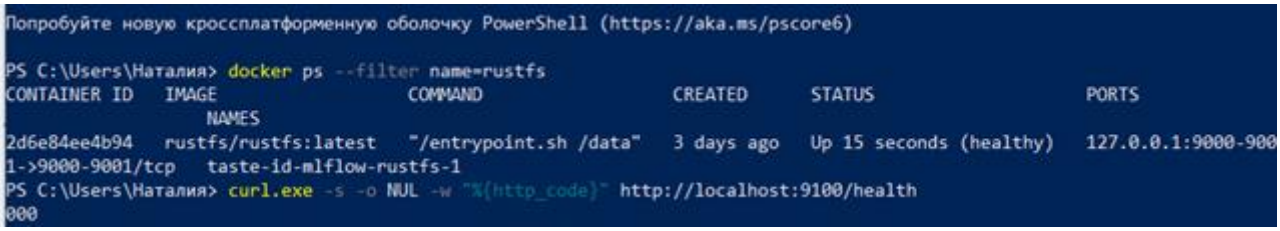


Рисунок 1 — Состояние контейнера rustfs (docker ps) и проверка эндпойнта /health: сервис ещё не отвечает (000)

Контейнер был остановлен после перерыва, поэтому запустила его командой docker start rustfs, после чего /health стал возвращать 200. Функцию aws3 (S3-клиент AWS CLI в Docker) в новой сессии PowerShell определила заново и создала бакет hh-lake.

```
docker start rustfs
function aws3 { docker run --rm -v C:\rustfs-hw\data:/data -e AWS_ACCESS_KEY_ID=rustfsadmin -e
AWS_SECRET_ACCESS_KEY=rustfsadmin -e AWS_DEFAULT_REGION=us-east-1 amazon/aws-cli --endpoint-url
http://host.docker.internal:9100 @args }
aws3 s3 mb s3://hh-lake
aws3 s3 ls
```

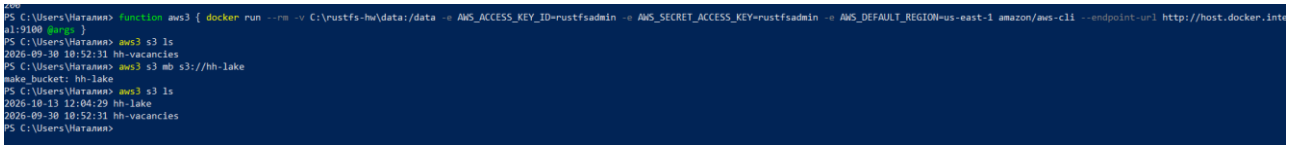


Рисунок 2 — Эндпойнт /health отвечает 200, функция aws3 определена, бакет hh-lake создан

1.2. Определение вакансий своего варианта (этап 2, подготовка).

Датасет it_vacancies_2024.csv содержит 229 388 записей и 23 колонки. Заголовок файла и начало одного из JSON-файлов показаны на рисунке 3 (иероглифы в выводе связаны с кодировкой консоли PowerShell и на данные не влияют). Колонка sphere пустая во всех строках, поэтому профессиональную группу определяла по колонке role.

```
Get-Content C:\rustfs-hw\data\vacancies\it_vacancies_2024.csv -TotalCount 1
Get-ChildItem C:\rustfs-hw\data\json1000 | Select-Object -First 1 | ForEach-Object { Get-
Content $_.FullName -TotalCount 60 }
```



Рисунок 3 — Заголовок CSV (колонки source … description, url) и начало JSON-файла

Вакансии варианта задала через role = «Программист, разработчик» и признак 1С в названии вакансии. Простая проверка «название содержит 1с или 1c» оказалась неточной, поэтому сравнила несколько условий (SQL-запросы к DuckDB по файлу в Docker):

Условие	Вакансий
role = «Программист, разработчик»	87 803
из них с «1с» или «1c» в названии (простой LIKE)	21 261
из них по регулярному выражению с границами «1с»	21 530
есть в простом условии, нет в регулярном	4
есть в регулярном, нет в простом (написание «1 С» через пробел)	273
в регулярном условии: вакансии «1С-Битрикс» / Bitrix	580

Из этих чисел следует три вывода. Простое условие теряет 273 вакансии, где название записано как «Программист 1 С», поэтому в выражении допускаю необязательный пробел между цифрой и буквой. Четыре вакансии, которые есть только в простом условии («1CERP», «1cfresh» и «1СБитрикс-разработчик»), в основном относятся к 1С, поэтому справа от «1с» границу не требую. Вакансии «1С-Битрикс» относятся к веб-разработке на CMS (PHP, JavaScript), а не к платформе 1С, поэтому исключила их по слову «битрикс» или «bitrix» в названии.

Итоговое условие варианта (одинаковое в SQL для DuckDB и в Python для MapReduce). Регулярное выражение: перед единицей не должно стоять другой цифры (чтобы не ловить «11с», «21с»), затем идёт «1», необязательный пробел и буква «с» латинская или «с» кириллическая.

```
role = 'Программист, разработчик'
AND regexp_matches(lower(name), '(^|[^0-9])1\s?[cс]')
AND NOT (lower(name) LIKE '%битрикс%' OR lower(name) LIKE '%bitrix%')
```

Результат проверки итогового условия:

Показатель	Значение
Вакансий варианта	20 953
из них с непустым key_skills (участвуют в подсчёте навыков)	13 452
Исключено вакансий «1С-Битрикс»	581
Не вошли: 1С есть только в навыках, но не в названии	1 369

Вакансии, где 1С указана только в навыках («Веб-программист», «Frontend-разработчик», «Программист С++» и подобные), в вариант не включила: это разработчики общего профиля, у которых 1С стоит среди прочих технологий.

1.3. Загрузка данных в бакет hh-lake (этап 2).

Файл с вакансиями уже лежал в бакете hh-vacancies (практическая работа № 2), поэтому скопировала его на стороне сервера в бакет hh-lake под ключ raw/it_vacancies_2024.csv и сверила размер с локальным файлом. Выбор вакансий варианта выполняют сами программы при чтении этого объекта.

```
aws3 s3 ls s3://hh-vacancies/csv/ --human-readable
aws3 s3 cp s3://hh-vacancies/csv/it_vacancies_2024.csv s3://hh-lake/raw/it_vacancies_2024.csv
aws3 s3 ls s3://hh-lake/raw/ --human-readable
Get-Item C:\rustfs-hw\data\vacancies\it_vacancies_2024.csv | Select-Object Length
```

```
PS C:\Users\Наталия> aws3 s3 ls s3://hh-vacancies/csv/ --human-readable
2026-09-30 11:32:17  808.4 MiB it_vacancies_2024.csv
PS C:\Users\Наталия> aws3 s3 cp s3://hh-vacancies/csv/it_vacancies_2024.csv s3://hh-lake/raw/it_vacancies_2024.csv
copy: s3://hh-vacancies/csv/it_vacancies_2024.csv to s3://hh-lake/raw/it_vacancies_2024.csv
PS C:\Users\Наталия> aws3 s3 ls s3://hh-lake/raw/ --human-readable
2026-10-13 12:22:25  808.4 MiB it_vacancies_2024.csv
PS C:\Users\Наталия> Get-Item C:\rustfs-hw\data\vacancies\it_vacancies_2024.csv | Select-Object Length

   Length
   ------
847619384
```

Рисунок 4 — Объект raw/it_vacancies_2024.csv (808,4 МиБ) в бакете hh-lake; размер локального файла 847 619 384 байт

1.4. Проверка чтения из кода: boto3 и DuckDB.

Весь код лежит в папке C:\rustfs-hw\hh-lake-hw и запускается в Docker (образ hhlake на python:3.12-slim с пакетами duckdb, boto3, psutil). Общие настройки (адрес хранилища, ключи, имя бакета, условие варианта) вынесены в src/config.py. Проверочный скрипт читает размер объекта и первые 200 байт через boto3 (Range-запрос), а затем через DuckDB (расширение httpfs) считает строки в том же объекте и вакансии варианта.

```
docker build -t hhlake .
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/test_s3.py
git --version
```

```
PS C:\rustfs-hw\hh-lake-hw> docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/test_s3.py
boto3: размер объекта = 847619384
boto3: первые 200 байт (Range-запрос) -> source,vacancy_id,status,snapshot_date,published_at,area,employer_id,employer_inn,employer_ogrn,employer_name,name,role,
DuckDB: всего строк в CSV: 229388
DuckDB: вакансий варианта: 20953
время, с: 141.4
PS C:\rustfs-hw\hh-lake-hw>
PS C:\rustfs-hw\hh-lake-hw> git --version
git version 2.46.2.windows.1
```

Рисунок 5 — Чтение объекта из hh-lake: размер 847 619 384 байт, 229 388 строк, 20 953 вакансии варианта; установлен Git 2.46.2

Число 20 953 стало контрольным для обоих способов. Два запроса DuckDB заняли 141,4 с, потому что при каждом обращении он заново скачивает объект (808 МиБ).

1.5. Собственная реализация MapReduce (этап 3, способ 1).

Программа src/mapreduce.py читает объект диапазонными запросами (Range) по 8 МиБ и не сохраняет файл на диск. Резать буфер по любому переводу строки нельзя: в колонке description переносы стоят внутри кавычек. Поэтому границей записи считается последний перевод строки с чётным числом кавычек перед ним, а недочитанный хвост переносится в следующий чанк. Получилось 102 чанка (847 619 384 = 101 × 8 388 608 + 369 976).

Map выполняют четыре процесса (ProcessPoolExecutor): каждый берёт один чанк, оставляет вакансии варианта, разбирает key_skills по символу «;», нормализует навыки (убирает пробелы, переводит в нижний регистр, заменяет латинскую «1c» на кириллическую «1с», потому что в данных встречаются оба написания, например «1С: ERP» и «1С: Предприятие 8»), берёт уникальные навыки вакансии и считает сочетания по 1, 2 и 3 навыка (itertools.combinations). Reduce суммирует частичные счётчики Counter в главном процессе. Если задача падает, чанк перезапускается (до трёх попыток); параметр --fail-chunk 5 принудительно роняет маппер на пятом чанке при первой попытке. Результаты записываются в results/mapreduce/skills_1.csv, skills_2.csv, skills_3.csv и загружаются в s3://hh-lake/results/mapreduce/.

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/mapreduce.py
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/mapreduce.py --fail-chunk 5 --no-upload
```

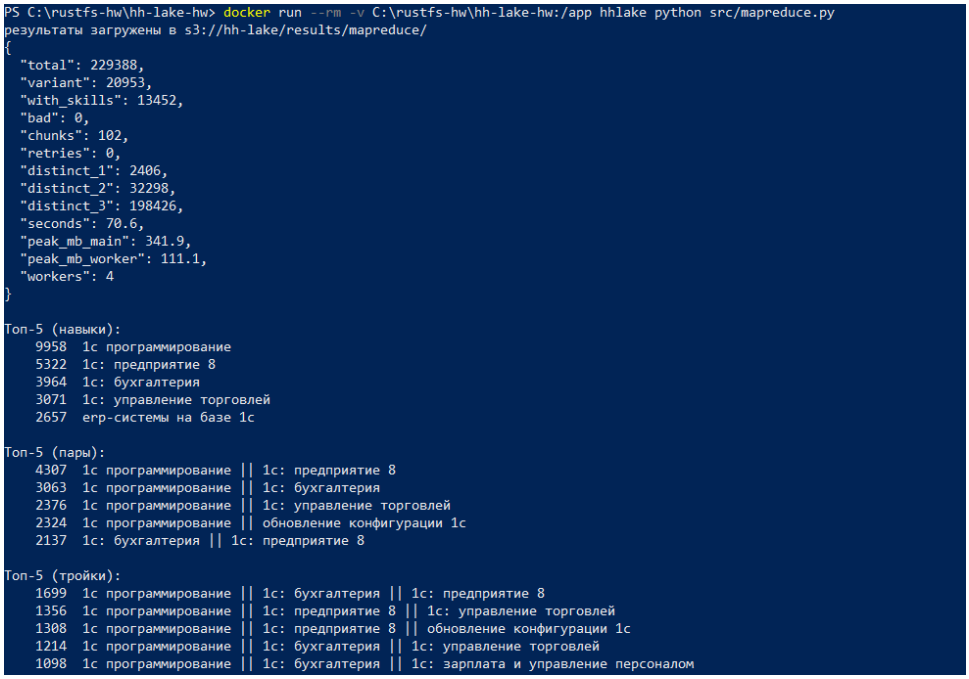


Рисунок 6 — Обычный запуск MapReduce: 229 388 записей, 20 953 вакансии варианта, 13 452 с навыками, 102 чанка, 70,6 с

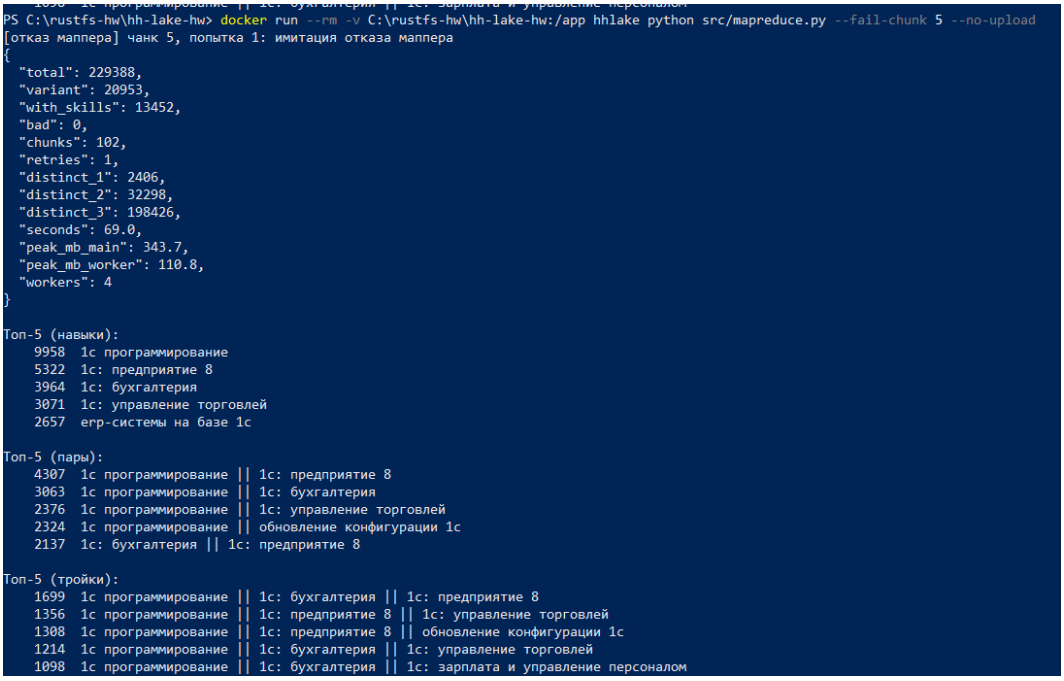


Рисунок 7 — Запуск с имитацией отказа маппера: чанк 5 перезапущен (retries = 1), итоговые числа те же

Показатель	Обычный запуск	С отказом маппера
Записей в файле / вакансий варианта	229 388 / 20 953	229 388 / 20 953
Вакансий с навыками	13 452	13 452
Битых строк	0	0
Чанков / повторных запусков	102 / 0	102 / 1
Уникальных навыков / пар / троек	2 406 / 32 298 / 198 426	2 406 / 32 298 / 198 426
Время	70,6 с	69,0 с
Пик памяти: главный процесс / воркер	341,9 / 111,1 МБ	343,7 / 110,8 МБ

Самые частые наборы навыков (число вакансий из 13 452):

Уровень	Вакансий	Набор навыков
Навыки	9 958	1с программирование

Уровень	Вакансий	Набор навыков
	5 322	1c: предприятие 8
	3 964	1c: бухгалтерия
Пары	4 307	1c программирование + 1c: предприятие 8
	3 063	1c программирование + 1c: бухгалтерия
	2 376	1c программирование + 1c: управление торговлей
Тройки	1 699	1c программирование + 1c: бухгалтерия + 1c: предприятие 8
	1 356	1c программирование + 1c: предприятие 8 + 1c: управление торговлей
	1 308	1c программирование + 1c: предприятие 8 + обновление конфигурации 1c

1.6. Подсчёт в DuckDB и верификация результатов (этап 3, способ 2 и этап 4).

Скрипт src/duckdb_skills.py читает тот же объект через httpfs. Условие варианта записано в SQL (см. 1.2), навыки нормализуются теми же правилами, что и в MapReduce. Из них строится временная таблица v (номер вакансии и отсортированный список её уникальных навыков), список разворачивается функцией unnest в таблицу u, а пары и тройки считаются самосоединением по номеру вакансии с условием a.skill < b.skill (чтобы не считать одну пару дважды):

```
CREATE TEMP TABLE u AS SELECT id, unnest(s) AS skill FROM v;

SELECT a.skill || ' || ' || b.skill AS skills, count(*) AS count
FROM u a JOIN u b ON a.id = b.id AND a.skill < b.skill
GROUP BY 1 ORDER BY 2 DESC, 1;

SELECT a.skill || ' || ' || b.skill || ' || ' || c.skill AS skills, count(*) AS count
FROM u a
JOIN u b ON a.id = b.id AND a.skill < b.skill
JOIN u c ON a.id = c.id AND b.skill < c.skill
GROUP BY 1 ORDER BY 2 DESC, 1;
```

Результаты DuckDB записываются в results/duckdb/ и загружаются в s3://hh-lake/results/duckdb/. Скрипт src/verify.py скачивает оба набора из бакета и сравнивает множества ключей и значение счётчика у каждого ключа.

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/duckdb_skills.py
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/verify.py
aws3 s3 ls s3://hh-lake/ --recursive --human-readable
```

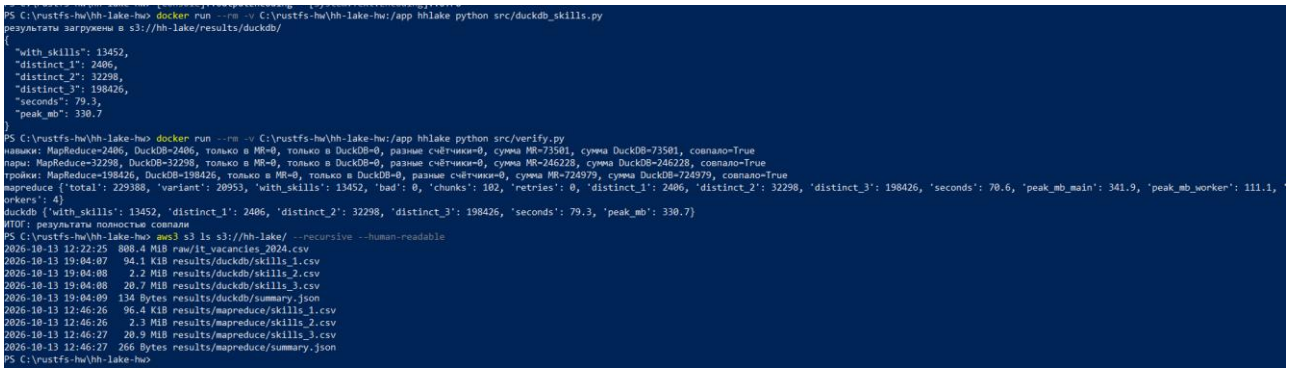


Рисунок 8 — DuckDB (13 452 вакансии с навыками, 79,3 с), сверка с MapReduce («совпало = True»), содержимое бакета hh-lake

Уровень	MapReduce, ключей	DuckDB, ключей	Различий в ключах и счётчиках	Сумма счётчиков MapReduce	Сумма счётчиков DuckDB	Совпало
Навыки	2 406	2 406	0	73 501	73 501	да
Пары	32 298	32 298	0	246 228	246 228	да
Тройки	198 426	198 426	0	724 979	724 979	да

Результаты MapReduce и DuckDB полностью совпали на всех трёх уровнях, число вакансий с навыками в обоих случаях 13 452. Небольшая разница в размерах файлов (например, skills_1.csv: 94,1 КиБ у DuckDB и 96,4 КиБ у MapReduce) объясняется символом конца строки: csv.writer в Python пишет \r\n, DuckDB пишет \n. Содержимое при этом совпадает.

1.7. Рост числа сочетаний: навыки, пары, тройки.

Размер набора	Уникальных наборов	Сумма вхождений	В среднем на вакансию
1 навык	2 406	73 501	5,46
2 навыка (пары)	32 298	246 228	18,30
3 навыка (тройки)	198 426	724 979	53,89

Уникальных пар в 13,4 раза больше, чем уникальных навыков, а уникальных троек в 82,5 раза больше. Причина в числе сочетаний внутри одной вакансии: при n уникальных навыках она даёт n одиночных навыков, $C(n, 2) = n(n-1)/2$ пар и $C(n, 3) = n(n-1)(n-2)/6$ троек. Тройки растут быстрее пар, потому что $C(n, 3)$ при больших n ведёт себя как n^3, а $C(n, 2)$ как n^2.

1.8. Влияние пар и троек навыков на медианную зарплату (этап 5).

Для анализа брала вакансии варианта с навыками и зарплатой в рублях (RUR): их 8 233 из 13 452. Зарплату вакансии определяла как середину вилки, если указаны обе границы, иначе брала ту границу, которая есть. Суммы «на руки» (gross = false) привела к сумме до вычета налога делением на 0,87, чтобы все значения были сопоставимы. Значения меньше 15 000 и больше 1 000 000 руб. отбросила как выбросы (осталось 8 222 вакансии), общая медиана равна 172 414 руб. Рассматривала только комбинации, которые встречаются не менее чем в 30 вакансиях с зарплатой: на меньшей выборке медиана ненадёжна.

Прибавку навыка считала от базового навыка: базовым берётся самый частый навык комбинации (по числу вакансий с зарплатой). Прибавка = медиана вакансий с комбинацией − медиана всех вакансий с базовым навыком. Сравнение с менее частым навыком давало бы искажённую оценку, поэтому так не делала.

Первый расчёт и обнаружение артефакта

В первом расчёте (рисунок 9) среди «понижающих» комбинаций десять (одна пара и девять троек) имели медиану ровно 20 000 руб., а у пар повторялись медианы 60 000 и 75 000 руб. Одинаковые круглые медианы у разных комбинаций из 30–70 вакансий выглядели подозрительно, поэтому проверила, откуда они берутся.

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/salary.py
```

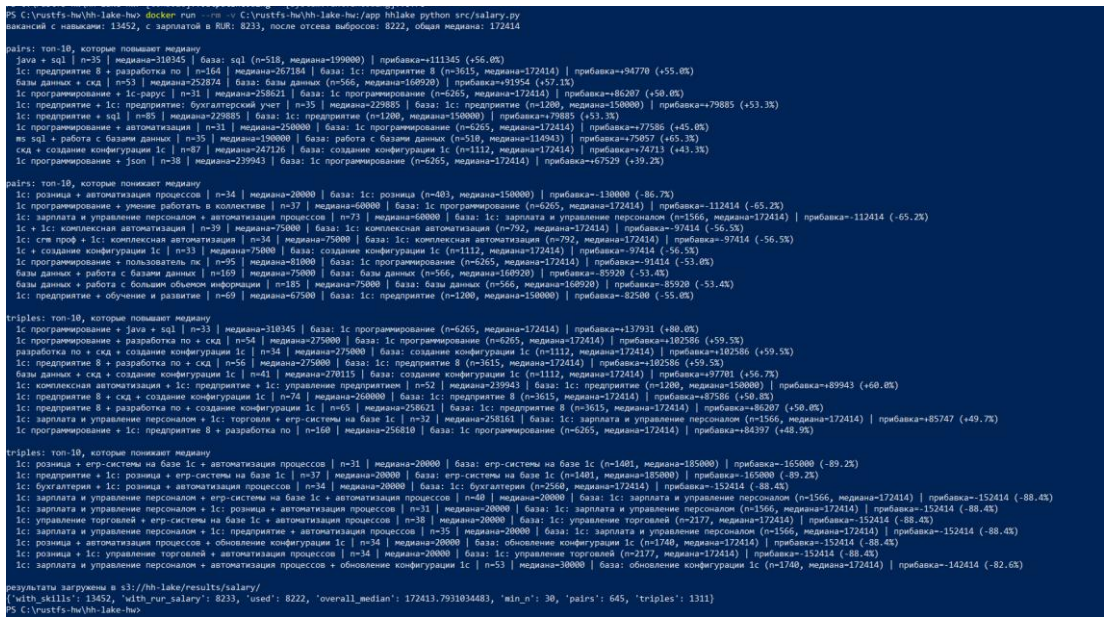


Рисунок 9 — Первый расчёт: топ-10 повышающих и понижающих пар и троек (до очистки данных)

Сначала добавила в расчёт число разных работодателей в каждой комбинации (рисунок 10; в данных employer_name пустая, поэтому работодателей различала по employer_id). У самой «дешёвой» пары 34 вакансии приходятся всего на 3 работодателей, у некоторых троек на 1–3. Но и после фильтра «не менее 5 работодателей» остались тройки с медианой 20 000 руб. (7 работодателей, 40 вакансий).

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/salary2.py
```

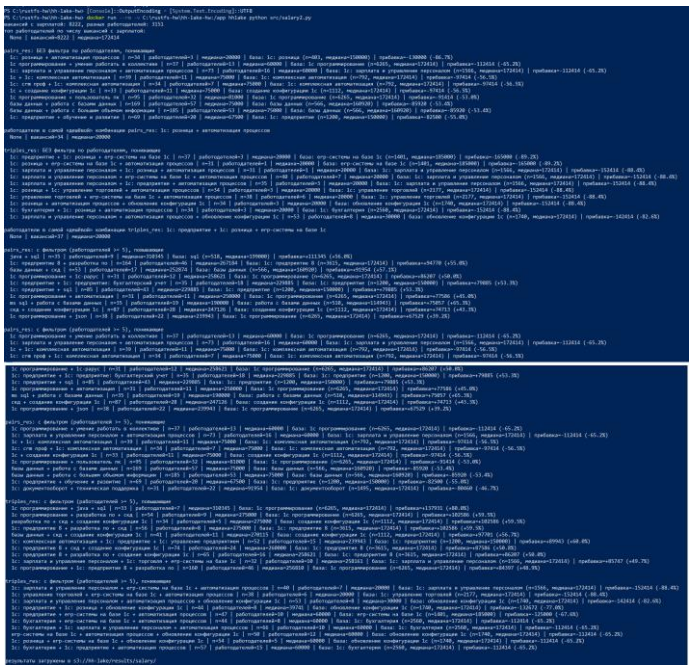


Рисунок 10 — Проверка по работодателям: число работодателей в комбинациях, пары и тройки после фильтра по числу работодателей

Затем посмотрела состав двух «дешёвых» групп: тройки с медианой 20 000 руб. и пары «1С: Зарплата и управление персоналом + автоматизация процессов» с медианой 60 000 руб..

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/salary_check.py
```

Показатель	Тройка (медиана 20 000)	Пара (медиана 60 000)
Вакансий / работодателей	40 / 7	73 / 16
Уникальных вакансий (работодатель, название, вилка)	11	28
Вакансий работодателя 3177	31	52
Самое частое название	Программист-стажёр 1С (Junior), 21	Программист-стажёр 1С (Junior), 21
Опыт «Нет опыта»	21	25

Для сравнения, по всей выборке с зарплатой (8 222 вакансии) зарплата ниже 30 000 руб. только у 1,3% вакансий и ниже 60 000 руб. у 8,1%. В «дешёвых» группах таких вакансий в разы больше, и почти все они от одного работодателя 3177: он массово публиковал одинаковые вакансии «Программист-стажёр 1С (Junior)» в июне 2024 года. Значит, низкая медиана этих комбинаций связана с этим работодателем, а не с навыками.

Очистка данных от дублей

Удалила полные дубли: вакансии с одинаковыми работодателем, названием, вилкой зарплаты, признаком gross, опытом и набором навыков оставила по одному разу. Выборка с зарплатой сократилась с 8 222 до 5 523 вакансий (удалено 2 699, около 33%), работодателей осталось 3 151, общая медиана не изменилась (172 414 руб.). Сильнее всего дублировались объявления крупных работодателей: у работодателя 3177 было 209 вакансий с зарплатой и только 104 уникальных объявления, у работодателя 735216 132 и 15 (рисунок 11).

Дополнительно ввела два фильтра: не менее 5 разных работодателей в комбинации и доля самого крупного работодателя не более 0,5. После удаления дублей ни одна из 433 пар и 803 троек эти

фильтры не нарушила (наибольшая доля крупнейшего работодателя 0,29), поэтому в итоговые таблицы вошли все комбинации с $n \geq 30$.

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/salary3.py
```



Рисунок 11 — Расчёт после очистки от дублей: крупнейшие работодатели, топ повышающих и понижающих пар и троек

Итоговые таблицы

В колонке «Работодателей» указано число разных работодателей среди вакансий комбинации. Прибавка показана в рублях и в процентах относительно медианы базового навыка.

Пары навыков, повышающие медианную зарплату:

Комбинация	n	Работ.	Медиана, руб.	База (n; медиана)	Прибавка
1с: предприятие + sql	69	43	229 885	1с: предприятие (752; 160 920)	+68 966 (+42,9%)
1с: предприятие 8 + оптимизация кода	39	27	229 885	1с: предприятие 8 (2 432; 170 000)	+59 885 (+35,2%)
1с: зарплата и кадры + 1с: предприятие: бухгалтерский учет	32	25	229 885	1с: зарплата и кадры (513; 172 414)	+57 471 (+33,3%)
1с программирование + 1с: управление холдингом	30	19	229 885	1с программирование (4 227; 172 414)	+57 471 (+33,3%)
ms sql + создание конфигурации 1с	46	24	229 885	создание конфигурации 1с (740; 172 414)	+57 471 (+33,3%)

Пары навыков, понижающие медианную зарплату:

Комбинация	n	Работ.	Медиана, руб.	База (n; медиана)	Прибавка
erp-системы на базе 1с + техническая поддержка	30	24	126 466	erp-системы на базе 1с (955; 201 149)	−74 684 (−37,1%)
администрирование + обновление конфигурации 1с	33	27	103 448	обновление конфигурации 1с (1 161; 172 414)	−68 966 (−40,0%)
1с: предприятие + техническая поддержка	38	30	94 828	1с: предприятие (752; 160 920)	−66 092 (−41,1%)
1с программирование + администрирование	63	48	109 195	1с программирование (4 227; 172 414)	−63 218 (−36,7%)
1с: предприятие + обучение и развитие	33	20	100 000	1с: предприятие (752; 160 920)	−60 920 (−37,9%)

Тройки навыков, повышающие медианную зарплату:

Комбинация	n	Работ.	Медиана, руб.	База (n; медиана)	Прибавка
1с программирование + erp-системы на базе 1с + sql	58	34	235 690	1с программирование (4 227; 172 414)	+63 276 (+36,7%)
1с: erp + 1с: предприятие 8 + erp-системы на базе 1с	41	29	230 000	1с: предприятие 8 (2 432; 170 000)	+60 000 (+35,3%)
1с: предприятие 8 + erp-системы на базе 1с + sql	42	24	229 943	1с: предприятие 8 (2 432; 170 000)	+59 943 (+35,3%)
1с: предприятие 8 + разработка по + создание конфигурации 1с	33	16	229 885	1с: предприятие 8 (2 432; 170 000)	+59 885 (+35,2%)
1с: предприятие 8 + скд + создание конфигурации 1с	42	24	229 885	1с: предприятие 8 (2 432; 170 000)	+59 885 (+35,2%)

Тройки навыков, понижающие медианную зарплату:

Комбинация	n	Работ.	Медиана, руб.	База (n; медиана)	Прибавка
базы данных + работа с базами данных + работа с большим объемом информации	34	23	114 943	базы данных (339; 172 414)	−57 471 (−33,3%)
1с программирование + создание конфигурации 1с + техническая поддержка	30	29	114 943	1с программирование (4 227; 172 414)	−57 471 (−33,3%)
1с: бухгалтерия + 1с: предприятие 8 + обучение и развитие	32	15	114 943	1с: предприятие 8 (2 432; 170 000)	−55 057 (−32,4%)
1с программирование + 1с: предприятие + техническая поддержка	30	25	123 471	1с программирование (4 227; 172 414)	−48 943 (−28,4%)
1с программирование + обновление конфигурации 1с + обучение и развитие	40	25	124 592	1с программирование (4 227; 172 414)	−47 822 (−27,7%)

Интерпретация

Медиану повышают навыки, которые отличают разработчика от пользователя или администратора: SQL (в разных формах: sql, ms sql, ms sql server), оптимизация кода, СКД, создание конфигураций, ERP-системы, БСП, управление холдингом. Это работа с отчётами, производительностью и базами данных, то есть более сложная разработка. Лучшая пара «1с: предприятие + sql» даёт медиану 229 885 руб., что на 68 966 руб. (42,9%) выше медианы базового навыка «1с: предприятие» (160 920 руб.). Лучшая тройка «1с программирование + erp-системы на базе 1с + sql» даёт 235 690 руб., на 63 276 руб. (36,7%) выше медианы навыка «1с программирование» (172 414 руб.).

Медиану понижают навыки, которые стоят рядом с программированием, но описывают другую работу: техническая поддержка, администрирование, консультирование, обучение и развитие, «продвинутый пользователь ПК», «знание 1С». Скорее всего, это вакансии первой линии поддержки и начальных позиций, где специалист работает с пользователями и сопровождением, а не с разработкой. Самая массовая из понижающих пар «1с программирование + техническая поддержка» (116 вакансий, 96 работодателей) даёт медиану 114 943 руб., на 33,3% ниже базовой. Это корреляция, а не причинность: часть эффекта объясняется уровнем позиции и опытом, которые отдельно не учитывались.

1.9. Региональный анализ (этап 6).

Выбрала три региона с наибольшим числом вакансий варианта: Москва, Санкт-Петербург и Екатеринбург. Зарплатные показатели считала по вакансиям с зарплатой в рублях после удаления дублей (методика из 1.8), показатели навыков по всем вакансиям варианта с навыками (13 452). Общее число вакансий варианта с навыками по всем регионам равно 13 452 и совпадает с результатом MapReduce.

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/regions.py
```

Основные запросы src/regions.py (b — вакансии варианта, us — развёрнутые навыки вакансий, vs — вакансии с зарплатой после удаления дублей; n_reg и n_all — число вакансий с навыками в регионе и во всех регионах):

```
CREATE TEMP TABLE vs AS SELECT id, area, e, sal FROM b
WHERE len(s) > 0 AND sal BETWEEN 15000 AND 1000000
QUALIFY row_number() OVER (PARTITION BY e, name, salary_from, salary_to, gross, experience, s
ORDER BY id) = 1;

SELECT count(*), round(median(sal)), round(quantile_cont(sal, 0.25)), round(quantile_cont(sal,
0.75))
FROM vs WHERE area = ?;

SELECT x.skill || ' + ' || y.skill, count(*) c
FROM us x JOIN us y ON x.id = y.id AND x.skill < y.skill
WHERE x.area = ? AND y.area = ? GROUP BY 1 ORDER BY c DESC, 1 LIMIT 7;

WITH reg AS (SELECT skill, count(*) c FROM us WHERE area = ? GROUP BY 1),
     allr AS (SELECT skill, count(*) c FROM us GROUP BY 1)
SELECT reg.skill, round(100.0 * reg.c / n_reg, 1), round(100.0 * allr.c / n_all, 1),
       round((1.0 * reg.c / n_reg) / (1.0 * allr.c / n_all), 2)
FROM reg JOIN allr USING (skill) WHERE reg.c >= 30 ORDER BY 4 DESC LIMIT 8;
```

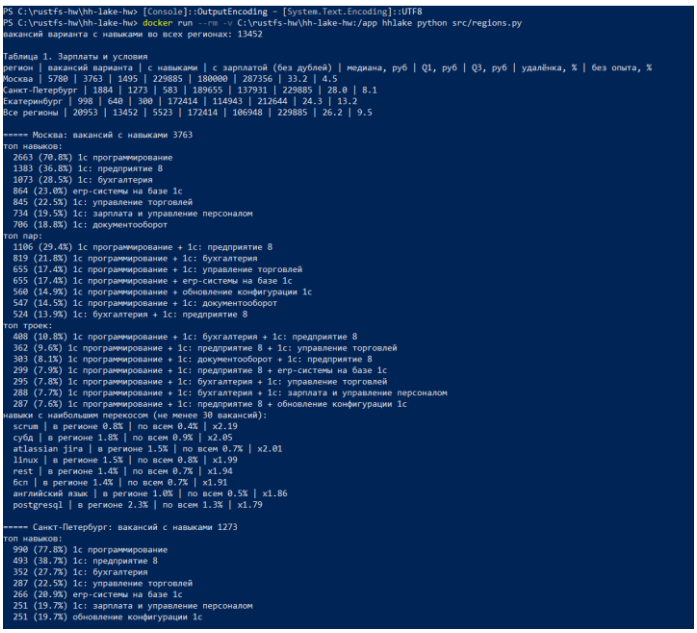


Рисунок 12 — Региональный анализ: вывод src/regions.py (таблица зарплат и условий, навыки, пары и тройки для Москвы и Санкт-Петербурга)

Таблица 1. Зарплаты и условия

Регион	Вакансий варианта	С навыками	С зарплатой (без дублей)	Медиана, руб.	Q1–Q3, тыс. руб.	Удалёнка	Без опыта
Москва	5 780	3 763	1 495	229 885	180,0–287,4	33,2%	4,5%
Санкт-Петербург	1 884	1 273	583	189 655	137,9–229,9	28,0%	8,1%
Екатеринбург	998	640	300	172 414	114,9–212,6	24,3%	13,2%
Все регионы	20 953	13 452	5 523	172 414	106,9–229,9	26,2%	9,5%

Таблица 2. Востребованность навыков, пар и троек (доля от вакансий с навыками региона)

Навыки (7 самых частых):

№	Москва (3 763)	Санкт-Петербург (1 273)	Екатеринбург (640)
1	1с программирование: 2 663 (70,8%)	1с программирование: 990 (77,8%)	1с программирование: 525 (82,0%)
2	1с: предприятие 8: 1 383 (36,8%)	1с: предприятие 8: 493 (38,7%)	1с: предприятие 8: 245 (38,3%)
3	1с: бухгалтерия: 1 073 (28,5%)	1с: бухгалтерия: 352 (27,7%)	1с: бухгалтерия: 187 (29,2%)
4	erp-системы на базе 1с: 864 (23,0%)	1с: управление торговлей: 287 (22,5%)	обновление конфигурации 1с: 129 (20,2%)
5	1с: управление торговлей: 845 (22,5%)	erp-системы на базе 1с: 266 (20,9%)	1с: управление торговлей: 127 (19,8%)
6	1с: зарплата и управление персоналом: 734 (19,5%)	1с: зарплата и управление персоналом: 251 (19,7%)	erp-системы на базе 1с: 119 (18,6%)

№	Москва (3 763)	Санкт-Петербург (1 273)	Екатеринбург (640)
7	1с: документооборот: 706 (18,8%)	обновление конфигурации 1с: 251 (19,7%)	1с: документооборот: 101 (15,8%)

Пары (5 самых частых):

№	Москва	Санкт-Петербург	Екатеринбург
1	1с программирование + 1с: предприятие 8: 1 106 (29,4%)	1с программирование + 1с: предприятие 8: 400 (31,4%)	1с программирование + 1с: предприятие 8: 217 (33,9%)
2	1с программирование + 1с: бухгалтерия: 819 (21,8%)	1с программирование + 1с: бухгалтерия: 276 (21,7%)	1с программирование + 1с: бухгалтерия: 162 (25,3%)
3	1с программирование + 1с: управление торговлей: 655 (17,4%)	1с программирование + 1с: управление торговлей: 230 (18,1%)	1с программирование + обновление конфигурации 1с: 118 (18,4%)
4	1с программирование + erp-системы на базе 1с: 655 (17,4%)	1с программирование + обновление конфигурации 1с: 219 (17,2%)	1с программирование + 1с: управление торговлей: 106 (16,6%)
5	1с программирование + обновление конфигурации 1с: 560 (14,9%)	1с программирование + erp-системы на базе 1с: 199 (15,6%)	1с программирование + erp-системы на базе 1с: 96 (15,0%)

Тройки (5 самых частых):

№	Москва	Санкт-Петербург	Екатеринбург
1	1с программирование + 1с: бухгалтерия + 1с: предприятие 8: 408 (10,8%)	1с программирование + 1с: бухгалтерия + 1с: предприятие 8: 146 (11,5%)	1с программирование + 1с: бухгалтерия + 1с: предприятие 8: 81 (12,7%)
2	1с программирование + 1с: предприятие 8 + 1с: управление торговлей: 362 (9,6%)	1с программирование + 1с: предприятие 8 + 1с: управление торговлей: 130 (10,2%)	1с программирование + 1с: предприятие 8 + обновление конфигурации 1с: 70 (10,9%)
3	1с программирование + 1с: документооборот + 1с: предприятие 8: 303 (8,1%)	1с программирование + 1с: предприятие 8 + обновление конфигурации 1с: 114 (9,0%)	1с программирование + 1с: предприятие 8 + 1с: управление торговлей: 65 (10,2%)
4	1с программирование + 1с: предприятие 8 + erp-системы на базе 1с: 299 (7,9%)	1с программирование + 1с: бухгалтерия + 1с: зарплата и управление персоналом: 113 (8,9%)	1с программирование + 1с: бухгалтерия + 1с: управление торговлей: 62 (9,7%)
5	1с программирование + 1с: бухгалтерия + 1с: управление торговлей: 295 (7,8%)	1с программирование + 1с: бухгалтерия + 1с: управление торговлей: 111 (8,7%)	1с программирование + 1с: бухгалтерия + 1с: зарплата и управление персоналом: 51 (8,0%)

Навыки с наибольшим региональным перекосом (отношение доли вакансий с навыком в регионе к доле по всем регионам; рассматривались навыки, которые встречаются не менее чем в 30 вакансиях региона):

Регион	Навык	В регионе, %	По всем, %	Перекос
Москва	scrum	0,8	0,4	×2,19
	субд	1,8	0,9	×2,05
	atlassian jira	1,5	0,7	×2,01
	linux	1,5	0,8	×1,99
	rest	1,4	0,7	×1,94
	бсп	1,4	0,7	×1,91
	английский язык	1,0	0,5	×1,86
	postgresql	2,3	1,3	×1,79
Санкт-Петербург	1с	5,0	3,3	×1,50
	оптимизация кода	2,6	1,8	×1,47
	git	2,4	1,7	×1,39
	постановка задач разработчикам	4,2	3,2	×1,31
	1с: erp	5,0	4,1	×1,24
	скд	4,3	3,6	×1,21

Регион	Навык	В регионе, %	По всем, %	Перекос
	1с: управление производственным предприятием	12,1	10,8	×1,12
	1с: управление предприятием	5,7	5,1	×1,10
Екатеринбург	разработка по	5,0	2,9	×1,71
	xml	5,0	3,4	×1,49
	работа с базами данных	7,2	5,3	×1,35
	аналитическое мышление	6,3	5,5	×1,14
	1с программирование	82,0	74,0	×1,11
	создание конфигурации 1с	13,0	12,3	×1,05
	1с: зарплата и кадры	8,9	8,7	×1,03
	обновление конфигурации 1с	20,2	19,7	×1,02

Наблюдения

Зарплата. Медиана в Москве (229 885 руб.) выше петербургской (189 655 руб.) на 21% и екатеринбургской (172 414 руб.) на 33%. Нижний квартиль Москвы (180 000 руб.) почти равен медиане Екатеринбурга, то есть сдвинулось всё распределение, а не отдельные дорогие вакансии. Выборка по Екатеринбургу небольшая (300 вакансий с зарплатой), поэтому его медиане доверяю меньше.

Условия. Чем ниже зарплата региона, тем меньше удалённой работы (33,2%, 28,0%, 24,3%) и тем больше вакансий без опыта (4,5%, 8,1%, 13,2%). Екатеринбург сильнее ориентирован на начинающих, что согласуется с самым низким нижним квартилем.

Навыки. Ядро во всех регионах одно: «1с программирование», «1с: предприятие 8», «1с: бухгалтерия», а самая частая пара «1с программирование + 1с: предприятие 8» встречается в 29,4%, 31,4% и 33,9% вакансий. В Москве чаще, чем в среднем, встречаются Scrum, Jira, СУБД, Linux, REST и PostgreSQL (в 1,8–2,2 раза), то есть командная разработка и интеграции. В Санкт-Петербурге выделяются оптимизация кода, Git и СКД, в Екатеринбурге «разработка ПО», XML и работа с базами данных, а пара «1с программирование + обновление конфигурации 1с» входит в тройку лидеров (18,4% против 14,9% в Москве), что похоже на сопровождение типовых конфигураций.

Это согласуется с этапом 5: родственные SQL и оптимизации кода навыки чаще встречаются в Москве и Санкт-Петербурге, где зарплаты выше. Причинную связь это не доказывает, потому что регионы различаются ещё и типом работодателей, который отдельно не рассматривался.

1.10. Профилирование: время и пиковая память.

Каждый подход запускала три раза, каждый запуск в новом контейнере. Скрипт src/profile.py запускает программу как дочерний процесс, измеряет время по часам и берёт пиковую память всего контейнера из cgroup (memory.peak), то есть суммарно по всем процессам. Для MapReduce запуск без загрузки результатов (--no-upload), для DuckDB полный запуск скрипта.

```
foreach ($m in "mapreduce","duckdb") { foreach ($i in 1..3) { docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/profile.py $m $i } }
```

```
PS C:\rustfs-hw\hh-lake-hw> [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
PS C:\rustfs-hw\hh-lake-hw> foreach ($m in "mapreduce","duckdb") { foreach ($i in 1..3) { docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/profile.py $m $i } }
mapreduce 1 81.4 513.2 0
mapreduce 2 80.9 523.0 0
mapreduce 3 78.9 521.7 0
duckdb 1 76.1 308.4 0
duckdb 2 84.4 343.4 0
duckdb 3 79.3 355.3 0
```

Рисунок 13 — Профилирование: три запуска MapReduce и три запуска DuckDB (метод, номер запуска, время в секундах, пик памяти в МБ, код завершения)

Подход	Запуск	Время, с	Пик памяти, МБ	Код завершения
MapReduce	1	81,4	513,2	0
	2	80,9	523,0	0

Подход	Запуск	Время, с	Пик памяти, МБ	Код завершения
	3	78,9	521,7	0
MapReduce, среднее		80,4	519,3	
DuckDB	1	76,1	308,4	0
	2	84,4	343,4	0
	3	79,3	355,3	0
DuckDB, среднее		79,9	335,7	

Время обоих подходов практически одинаковое (около 80 с), хотя устроены они по-разному. По памяти DuckDB экономнее: в среднем 335,7 МБ против 519,3 МБ у MapReduce (меньше на 35%). У MapReduce память складывается из буферов чанков в четырёх процессах и словарей Counter с сотнями тысяч троек, которые накапливаются в каждом процессе и затем сливаются в главном. Одинаковое время наводит на мысль, что оба подхода упираются в чтение объекта (проверено в 1.11).

1.11. Бонус: оптимизация MapReduce, сравнение «до» и «после».

Если время упирается в чтение, его можно проверить отдельно. Скрипт src/bench_io.py скачивает весь объект диапазонами по 8 МиБ без какой-либо обработки, сначала одним потоком, затем в 4, 8 и 16 потоков; последняя строка (снова один поток) показывает, что файловый кэш на результат не влияет.

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/bench_io.py
```

```
PS C:\rustfs-hw\hh-lake-hw> [Console]::OutputEncoding = [System.Text.Encoding]::UTF8
PS C:\rustfs-hw\hh-lake-hw> docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/bench_io.py
потоков=1: 76.1 с, 10.6 МиБ/с, байт=847619384
потоков=4: 21.5 с, 37.6 МиБ/с, байт=847619384
потоков=8: 17.8 с, 45.5 МиБ/с, байт=847619384
потоков=16: 16.9 с, 47.8 МиБ/с, байт=847619384
потоков=1: 74.0 с, 10.9 МиБ/с, байт=847619384
PS C:\rustfs-hw\hh-lake-hw>
```

Рисунок 14 — Скорость чтения объекта 847 619 384 байт из RustFS в 1, 4, 8, 16 потоков

Потоков	Время, с	Скорость, МиБ/с	Ускорение к 1 потоку
1	76,1	10,6	1,0
4	21,5	37,6	3,5
8	17,8	45,5	4,3
16	16,9	47,8	4,5
1 (повтор)	74,0	10,9	1,0

Одним потоком объект читается за 76 с (10,6 МиБ/с), что почти равно полному времени MapReduce и DuckDB. Прирост от 8 до 16 потоков небольшой, поэтому в оптимизированной версии выбрала 8 потоков. Что изменено в src/mapreduce_opt.py:

1. Диапазоны по 8 МиБ скачиваются заранее в 8 потоках, а в обработку выдаются строго по порядку, чтобы границы записей определялись так же, как раньше.
2. Главный процесс не декодирует текст чанка, он передаёт воркеру сырые байты, а декодирует их сам воркер.
3. Поиск границы записи упрощён: вместо разбиения всего буфера по кавычкам ищется последний перевод строки, и число кавычек до него считается методом bytes.count.

Логика отбора вакансий, нормализации и подсчёта осталась прежней. Скрипт src/verify_opt.py сравнил результаты оптимизированной версии с исходным MapReduce и с DuckDB по всем трём уровням (шесть сравнений), и все они совпали; затем сделала три замера теми же средствами, что и «до».

```
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/mapreduce_opt.py
docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python src/verify_opt.py
foreach ($i in 1..3) { docker run --rm -v C:\rustfs-hw\hh-lake-hw:/app hhlake python
src/profile.py mapreduce_opt $i }
```

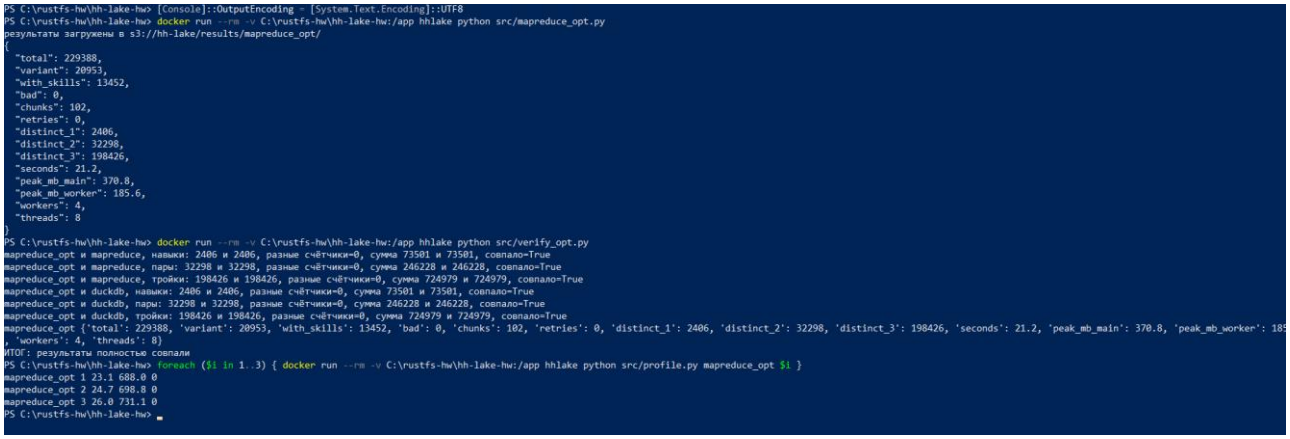


Рисунок 15 — Оптимизированный MapReduce: 21,2 с, результаты совпали с MapReduce и DuckDB, три замера профилирования

Версия	Запуск 1	Запуск 2	Запуск 3	Среднее
MapReduce «до»: время, с	81,4	80,9	78,9	80,4
MapReduce «после»: время, с	23,1	24,7	26,0	24,6
MapReduce «до»: пик памяти, МБ	513,2	523,0	521,7	519,3
MapReduce «после»: пик памяти, МБ	688,0	698,8	731,1	706,0

Среднее время MapReduce сократилось с 80,4 до 24,6 с, то есть в 3,3 раза, и он стал быстрее DuckDB примерно в 3,2 раза. Платой стала память: пик вырос с 519,3 до 706,0 МБ (на 36%), потому что восемь потоков одновременно держат скачанные диапазоны по 8 МиБ, а воркерам передаются сырые байты. По памяти DuckDB остаётся самым экономным (335,7 МБ), а собственный MapReduce даёт больше контроля над чтением, повтором упавших чанков и форматом результатов.

1.12. Репозиторий GitHub (этап 7).

Исходный код всех программ и данный отчёт опубликованы в репозитории:

https://github.com/nataliaaa23/hh-lake-1c

В папке src лежат все программы, в папке results сводки и таблицы. Большие файлы skills_*.csv в репозиторий не включены, они находятся в бакете hh-lake (results/mapreduce, results/duckdb).

Выводы

1. Файл с вакансиями загружен в бакет hh-lake в RustFS. Вакансии варианта «Программист, разработчик на 1С» отобраны по полю role и признаку 1С в названии (без «1С-Битрикс»): 20 953 вакансии, из них 13 452 с навыками.

2. Собственный MapReduce (102 чанка по 8 МиБ из S3, 4 процесса, повтор упавшего чанка) и DuckDB дали полностью совпавшие результаты: 2 406 навыков, 32 298 пар и 198 426 троек; тест с отказом маппера прошёл.

3. Уникальных пар и троек в 13,4 и 82,5 раза больше, чем навыков, из-за сочетаний C(n, 2) и C(n, 3) внутри вакансии.

4. Медиану повышают SQL, оптимизация кода, СКД, создание конфигураций и ERP (до +42,9%), понижают техническая поддержка, администрирование и обучение (до −41,1%). Медиана 20 000 руб. в первом расчёте была артефактом дублей одного работодателя.

5. Москва по медиане (229 885 руб.) опережает Санкт-Петербург (189 655 руб.) и Екатеринбург (172 414 руб.); чем ниже зарплата региона, тем меньше удалёнки и больше вакансий без опыта.

6. MapReduce и DuckDB работают около 80 с, DuckDB расходует меньше памяти (335,7 против 519,3 МБ). Загрузка в 8 потоков ускорила MapReduce в 3,3 раза (24,6 с) ценой роста памяти на 36%.